

Auto-fill połączeń przy imporcie szablonu workflow

AutomationService — importAutomation() — sembot-laravel

DATA	KOMPONENT	DOTYCZY	TYP ZMIANY
2026-04-23	AutomationService.php	sembot-laravel	Feature / UX improvement

1. Problem — stan przed zmianą

Przy imporcie szablonu workflow użytkownik musiał ręcznie konfigurować **każdy węzeł** wymagający połączenia z zewnętrznym serwisem (Google Ads, Microsoft Advertising, Google Merchant Center, GA4). Szablony przechowują te wartości jako `null`.

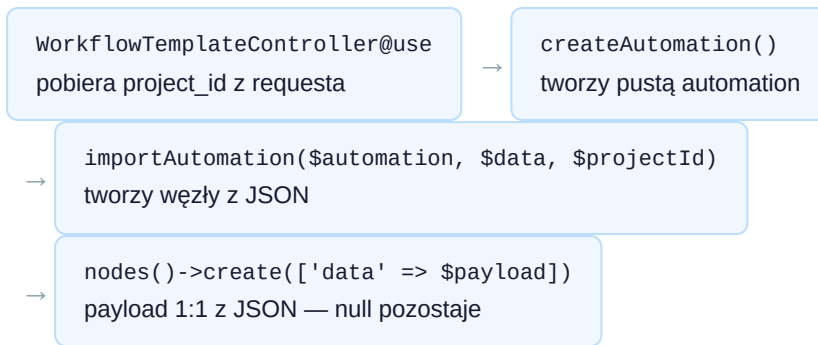
ZGŁOSZONY OBJAW

Po zaimportowaniu szablonu workflow (np. `budget_protector`) wszystkie węzły getter mają `connection_id: null` i `project_id: null`. Użytkownik musi wejść w każdy węzeł, wybrać połączenie i projekt z list rozwijanych, a dopiero potem zapisać — dla każdego węzła osobno.

Struktura payload węzła w szablonie JSON

```
{
  "type": "get_google_campaign",
  "data": {
    "connection_id": null,    // ← zawsze null w szablonach
    "project_id": null,      // ← zawsze null w szablonach
    "campaign_statuses": ["ENABLED"]
  }
}
```

Przepływ importu szablonu (przed)



Przyczyna techniczna

- 1 \$projectId nie był dostępny w transakcji DB**
Metoda `importAutomation()` przyjmuje parametr `int $projectId`, ale domknięcie `DB::transaction(function () use ($automation, $importData) {...})` nie zawierało `$projectId` w klauzuli `use`. Projekt był synchronizowany dopiero na końcu, poza transakcją.
- 2 Brak jakiejkolwiek logiki auto-fill**
W pierwszym przebiegu tworzenia węzłów payload był kopiowany 1:1 z JSON szablonu bez żadnych modyfikacji. Nie było mechanizmu pobierającego dostępne połączenia projektu ani przypisującego je do węzłów.

2. Rozwiązanie

Zmiana w jednym pliku: `app/Services/Automations/AutomationService.php`

Zmiana 1 — \$projectId w clocsurze transakcji

FIX

```
- DB::transaction(function () use ($automation, $importData) {  
+ DB::transaction(function () use ($automation, $importData, $projectId) {
```

Zmiana 2 — Pobranie połączeń przed pętlą + resolveNodeConnections()

NOWY KOD

```
// Przed pierwszym przebiegiem – 1 zapytanie do DB  
+ $latestConnectionsBySlug = $this->getLatestConnectionsBySlug($projectId);  
  
// W pierwszym przebiegu – payload rozwiązywany przed zapisem  
$newNode = $automation->nodes()->create([  
    'type' => $nodeData['type'],  
-    'data' => $nodeData['payload'],
```

```
+ 'data' => $this->resolveNodeConnections($nodeData['payload'], $projectId, $latestConnectionsBySlug);

    'parent_id' => null,
]);
```

Nowa metoda — getLatestConnectionsBySlug()

NOWA METODA PRYWATNA

```
/** @return array<string, ProjectConnection> */
private function getLatestConnectionsBySlug(int $projectId): array
{
    return ProjectConnection::where('project_id', $projectId)
        ->whereNull('deleted_at')
        ->orderByDesc('id')           // najnowszy pierwszy
        ->get()
        ->unique('external_service_slug') // jeden na slug
        ->keyBy(fn(ProjectConnection $pc) => $pc->external_service_slug
            instanceof ConnectionServiceSlug
                ? $pc->external_service_slug->value
                : (string) $pc->external_service_slug)
        ->all();
}
```

Nowa metoda — resolveNodeConnections()

NOWA METODA PRYWATNA

```
private function resolveNodeConnections(
    array $payload,
    int $projectId,
    array $latestConnectionsBySlug
): array {
    $nodeType = $payload['type'] ?? null;

    if (!isset($payload['data']) || !is_array($payload['data'])) {
        return $payload;
    }

    // Auto-fill project_id jeśli null
    if (array_key_exists('project_id', $payload['data']) && empty($payload['data']['project_id'])) {
        $payload['data']['project_id'] = $projectId;
    }

    // Auto-fill connection_id jeśli null i typ węzła ma znany slug
    if ($nodeType && array_key_exists('connection_id', $payload['data']) && empty($payload['data']['connection_id'])) {
        $slug = self::NODE_TYPE_CONNECTION_SLUG[$nodeType] ?? null;
        if ($slug !== null) {
            $connection = $latestConnectionsBySlug[$slug->value] ?? null;
        }
    }
}
```

```

        if ($connection != null) {
            $payload['data']['connection_id'] = $connection->id;
        }
    }
}

return $payload;
}

```

WAŻNA UWAGA — CONNECTION_ID TO PROJECTS_CONNECTIONS.ID, NIE CONNECTIONS.ID

Węzły w payload przechowują `connection_id` jako klucz główny tabeli pivot `projects_connections.id`, a nie `connections.id`. Wynika to z logiki `getCommand()` w klasach getterów: `$project->connections()->wherePivot('id', $this->connectionId)`.

Mapa typów węzłów → ConnectionServiceSlug

Stała `NODE_TYPE_CONNECTION_SLUG` zawiera mapowanie 36 typów węzłów:

Typ węzła	ConnectionServiceSlug
<code>get_google_campaign</code> , <code>get_google_ads_performance</code> , <code>get_google_ads_customer</code> , <code>get_google_ads_metrics</code> , <code>get_google_ads_ad_copy</code> , <code>get_ads_ad_groups</code> , <code>add_negative_keywords</code>	<code>GOOGLE_ADWORDS</code>
<code>get_microsoft_advertising_campaigns</code> , <code>get_microsoft_advertising_keywords</code> , <code>get_microsoft_advertising_search_terms</code> , <code>get_microsoft_advertising_ad_groups</code> , <code>get_microsoft_advertising_products</code>	<code>MICROSOFT_ADVERTISING</code>
<code>get_google_merchant_center_performance_info</code> , <code>get_google_merchant_center_price_competitiveness_info</code> , <code>get_google_merchant_center_competitive_landscape</code> , <code>get_google_merchant_center_top_brands</code> , <code>get_google_merchant_center_top_categories</code> , <code>get_google_merchant_center_top_products</code> , <code>get_google_merchant_center_shopping_trends_info</code>	<code>GOOGLE_MERCHANT_CENTER</code>
<code>get_google_analytics_4_audience_name_info</code> , <code>get_google_analytics_4_campaign_name_info</code> , <code>get_google_analytics_4_event_name_info</code> , <code>get_google_analytics_4_item_brand_info</code> , <code>get_google_analytics_4_item_name_info</code> , <code>get_google_analytics_4_new_vs_returning_info</code> , <code>get_google_analytics_4_source_medium_info</code> ,	<code>GOOGLE_ANALYTICS_4</code>

```
get_google_analytics_4_user_age_bracket_info ,  
get_google_analytics_4_user_gender_info
```

3. Wyniki testu

Test wykonany na projekcie 158 z połączeniami: `GOOGLE_ADWORDS (id=23)` , `MICROSOFT_ADVERTISING (id=18)` , `SHOPIFY_ADMIN (id=19)` . Import szablonu `budget_protector` .

Typ węzła	Node type	connection_id	project_id
trigger	schedule	brak pola	brak pola
getter	get_google_campaign	23 ✓ auto-fill	158 ✓ auto-fill
flow_control	iterator	brak pola	brak pola
getter	get_google_ads_metrics	23 ✓ auto-fill	158 ✓ auto-fill
flow_control	filter	brak pola	brak pola
action	create_task	brak pola	158 ✓ auto-fill

ZACHOWANIE

Węzły bez pól `connection_id` / `project_id` w payload (schedule, iterator, filter) pozostają bez zmian. Węzły z tymi polami ustawionymi na `null` dostają auto-fill. Jeśli użytkownik nie ma żadnego połączenia danego typu — pole pozostaje `null` , węzeł trafia do stanu `NEEDS_CONFIG` .

4. Podsumowanie zmian

Zmiana	Lokalizacja	Opis
Closure use	<code>importAutomation()</code> linia 368	Dodano <code>\$projectId</code> do klauzuli <code>use</code>
Auto-fill call	<code>importAutomation()</code> linia 393, 400	Wywołanie <code>getLatestConnectionsBySlug()</code> i <code>resolveNodeConnections()</code> w pierwszym przebiegu
Nowa metoda	linia 479	<code>getLatestConnectionsBySlug(int \$projectId)</code> — jedno zapytanie do DB per import

Nowa stała	linia 491	<code>NODE_TYPE_CONNECTION_SLUG</code> — mapa 36 typów węzłów → slug
------------	-----------	--

Nowa metoda	linia 531	<code>resolveNodeConnections()</code> — ustawia <code>project_id</code> i <code>connection_id</code> dla każdego węzła
-------------	-----------	--

DUPLIKOWANIE WORKFLOW

Funkcja duplikowania workflow nie istnieje jeszcze w systemie (brak endpointu). Gdy zostanie dodana i użyje `importAutomation()`, auto-fill zadziała automatycznie bez dodatkowych zmian.